



K L Deemed to be University
(Koneru Lakshmaiah Education Foundation)

GREEN FIELDS, VADDESWARAM · TERM 3, AY 2025-2026 · PROJECT-BASED LEARNING

MID-TERM EXAMINATION · PAPER SET 3

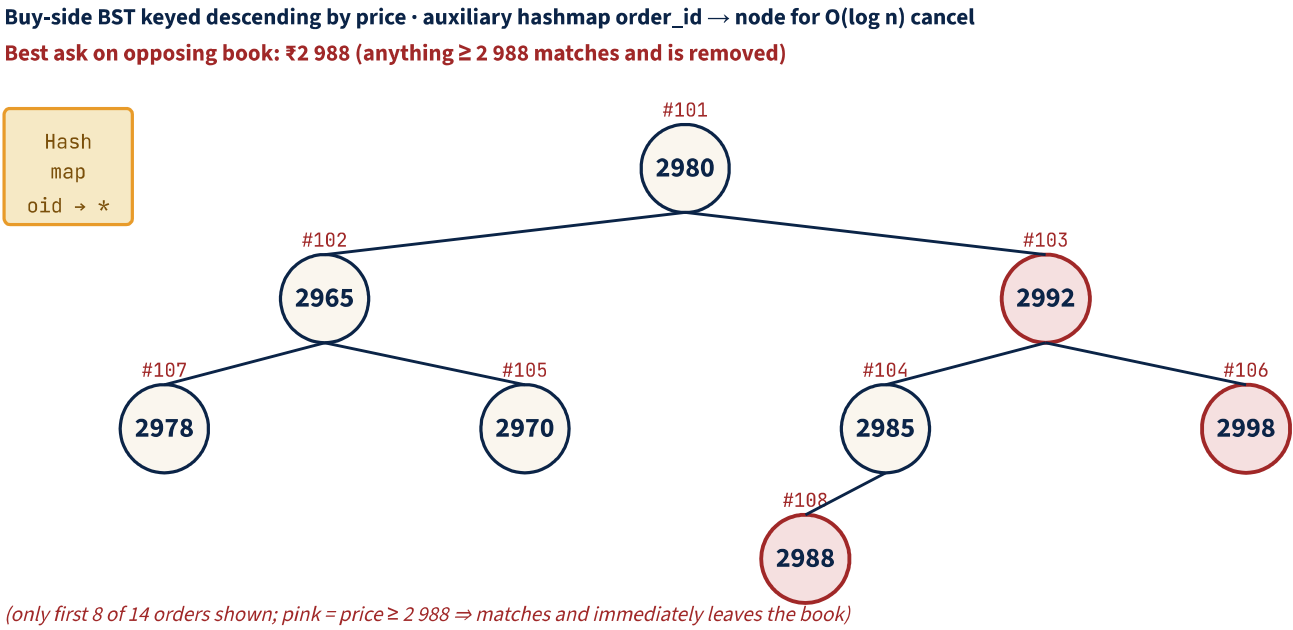
COURSE	DATA STRUCTURES AND ALGORITHMS - 2	CODE	25SC1305E
EXAMINATION	Mid-Term Examination	PAPER	Set 3
DURATION	90 minutes	MAX. MARKS	45
COORDINATOR	Zeelan Basha CMAK	MODE	Project-Based Learning

INSTRUCTIONS TO CANDIDATES

- Answer ALL three questions. Each question carries 15 marks. Maximum: 45 marks.
- Each question is a scenario / case-study / micro-project covering one Course Outcome.
- Show all working, intermediate steps, code listings, diagrams, derivations and reasoning traces.
- Time allowed: 90 minutes. Calculator allowed. No mobile phones, smart watches, or AI assistants.

SCENARIO / CASE STUDY

NSE cash-market order book — high churn with cancel-by-order-id. The buy-side order book for a single high-volume stock (think RELIANCE.NS) is keyed by limit price (descending: highest bid first). Behaviour observed at market open: **14 BUY orders arrive in the first 200 ms**. Of every batch like this, ~50 % are matched against the sell side *almost immediately* and removed; ~30 % rest in the book longer than 1 second; ~20 % are explicitly cancelled by the trader (cancel arrives carrying the original order_id, NOT the price). The matching engine must support: (i) insert(order), (ii) peek_best_bid() in O(1), (iii) delete_by_order_id(order_id) in O(log n) — the cancellation latency SLA is 50 μs at p99. The order list arriving in the first 200 ms (price in ₹, volume omitted): order 101 @ 2980, 102 @ 2965, 103 @ 2992, 104 @ 2985, 105 @ 2970, 106 @ 2998, 107 @ 2978, 108 @ 2988, 109 @ 2982, 110 @ 2995, 111 @ 2972, 112 @ 2990, 113 @ 2986, 114 @ 2975. Best ask on opposing book is currently 2 988.



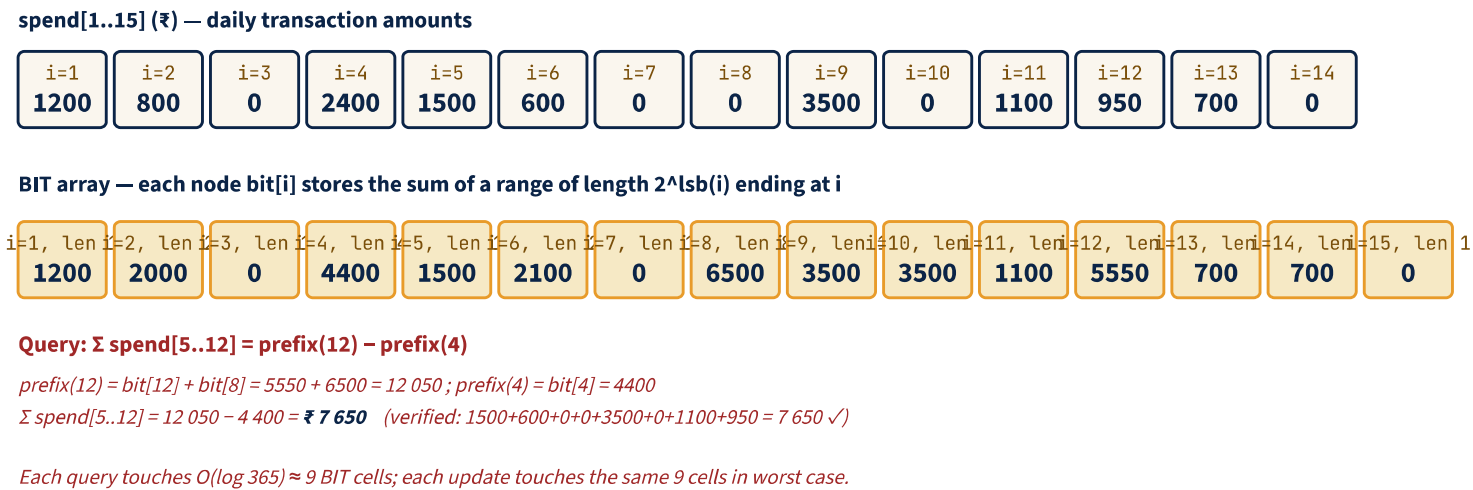
SUB	QUESTION	MARKS	CO	BTL
(a)	From the 14 incoming orders, identify which orders match against the opposing book at price ≥ 2 988 and are immediately removed. List them in arrival order. For the <i>remaining</i> orders that survive, build the BUY-side BST keyed in descending price (so root is highest bid; left child > parent > right child). Show the final tree.	5	C01	BTL3
Then quantify the difference between a plain BST and an AVL <i>for this workload's first 200 ms</i> : how many rotations does the AVL incur during construction, and how many of those rotations are <i>wasted work</i> because the corresponding nodes get deleted within the same second by either matching or cancellation? (Assume worst case: all 50 % matches happen, then all 20 % cancellations.)				
(b)	Complete the boilerplate below so that insert_order maintains <i>both</i> the price-keyed BST <i>and</i> a {order_id → node} hashmap, enabling O(log n) cancel-by-id later. Then complete cancel_by_id using the hashmap. (You do NOT need to implement BST rebalancing — assume an underlying balanced-BST library; focus on the dual-structure synchronisation.) <pre>class OrderNode { long orderId; double price; int vol; OrderNode left, right; OrderNode(long oid, double price, int vol) { this.orderId = oid; this.price = price; this.vol = vol; } } class OrderBook { OrderNode root = null; Map<Long, OrderNode> byId = new HashMap<>(); // orderId -> node void insertOrder(long oid, double price, int vol) { // TODO 1: insert into descending-price BST AND register in byId } boolean cancelById(long oid) { // TODO 2: O(1) lookup in byId, then O(log n) delete from BST, // then O(1) hashmap removal. Return true on success. return false; } OrderNode peekBestBid() { // Best bid is the LEFTMOST node (since tree is keyed descending, // left child > parent — so leftmost = highest price). // TODO 3: traverse leftmost path in O(log n). return null; } }</pre>	6	C01	BTL3
(c)	AVL or Red-Black? Both give O(log n) ops. For a workload where 50 % of inserts are followed by a delete <i>within the same second</i> , the rotation count matters: AVL maintains stricter balance (≤ 1 height-difference) and consequently performs more rotations on average. RB-	4	C01	BTL5

SUB	QUESTION	MARKS	CO	BTL
	trees allow sloppier balance (max 2× height) and incur fewer rotations per delete.			
	Argue quantitatively which is preferable for the order-book workload. Cite (i) the published rotation-count bounds for insert and delete in each tree (you may state them; no derivation needed), (ii) the consequence for the matching-engine's 50 μs cancellation SLA at 14 orders / 200 ms sustained, and (iii) one secondary consideration (cache-locality, code complexity, or worst-case vs amortised cost).			

SCENARIO / CASE STUDY

HDFC NetBanking — daily-spend prefix sums for monthly statements. For each customer, the system maintains an integer array `spend[1..365]` of daily-spend amounts (in ₹) for the calendar year. **Two operations dominate:** (i) *point update* — when a transaction posts, increment `spend[ today ]` by the amount (~50 such updates/customer/day during peak hours); (ii) *range-sum query* — when generating the monthly statement, compute total spend over `[ first_day_of_month, last_day_of_month ]` (~10 queries/customer at end of month, but called for 50 M customers in a 6-hour batch window). Importantly: **NO range updates** are needed — every transaction is a single point. Required: $O(\log n)$ for both ops; minimum memory; sequential cache access ideal for batch processing.

Today's input — first 15 days of January (₹): `spend = [0, 1200, 800, 0, 2400, 1500, 600, 0, 0, 3500, 0, 1100, 950, 700, 0]` (index 0 unused; indices 1..15 correspond to Jan 1..15). The query of interest: total spend over Jan 5–12.

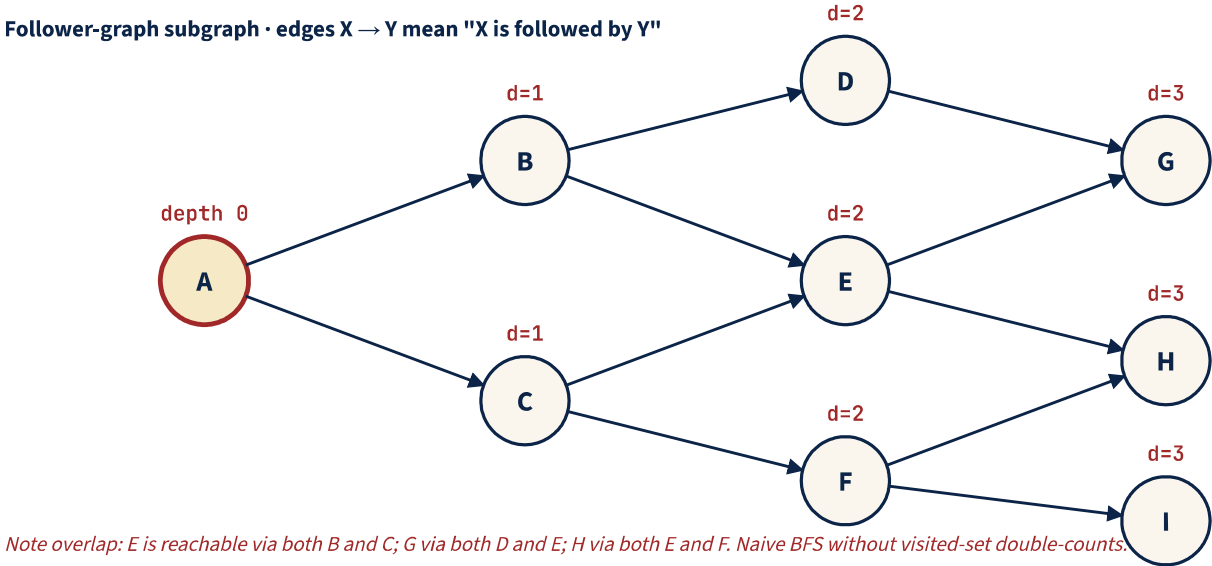


SUB	QUESTION	MARKS	CO	BTL
(a)	Construct the Fenwick (BIT) array for <code>spend[1..15]</code> as given in the scenario. Show your work for at least three non-trivial BIT cells (e.g., <code>bit[4]</code> , <code>bit[8]</code> , <code>bit[12]</code>) — for each, list the array indices it covers and verify the stored sum. Then use the BIT to answer the scenario's query (total spend Jan 5–12) by computing <code>prefix(12) - prefix(4)</code> . Show the BIT cells visited during each prefix call.	5	C02	BTL3
(b)	An engineer's <code>prefix_sum(bit, i)</code> implementation is shown below. It is intended to return the prefix sum <code>spend[1] + spend[2] + ... + spend[i]</code> . There is one off-by-one bug that produces wrong results for some values of <code>i</code> but correct results for others. Identify the bug, state which inputs trigger it, and provide the one-character fix. Also state the time-complexity (correct or buggy). <pre>class FenwickBuggy { int[] bit; // 1-indexed; bit[0] unused int n; FenwickBuggy(int n) { this.n = n; this.bit = new int[n + 1]; } /** Return spend[1] + spend[2] + ... + spend[i]. */ long prefixSum(int i) { long s = 0; while (i > 0) { s += bit[i]; i -= (i & -i) + 1; // (lsb removal, intended) } return s; } /** Add `delta` to the element at index i. */ void pointUpdate(int i, int delta) { while (i <= n) { bit[i] += delta; i += i & -i; } } }</pre>	6	C02	BTL4
(c)	An architect proposes replacing the BIT with a segment tree on the grounds that it 'is more general'. Argue precisely why <i>for this specific workload</i> the BIT is the better choice. Be quantitative: (i) memory comparison (BIT uses n words; segment tree uses $\sim 4n$), (ii) cache behaviour during the 50-million-customer batch, (iii) code surface (lines of working code). Then state two scenarios that, if true, would <i>flip</i> your recommendation toward segment tree.	4	C02	BTL5

SCENARIO / CASE STUDY

X (Twitter) — retweet-reach prediction. When a tweet is posted, the platform must predict its 10-minute reach for downstream services (ad-pricing, trending-topic detection). The prediction model uses bounded BFS on the follower graph: from the original author, traverse outward via 'is followed by' edges up to **depth 3**; deeper hops have empirically negligible retweet probability and are ignored. Important: in real follower graphs, many users follow each other (overlapping audiences), so a naive BFS will count the same user multiple times. Compute budget: **500 ms per tweet** at p99.

Today's miniature example for tracing: a small connected-component subgraph of 9 users is shown. Author is user **A** (the source). Edge $X \rightarrow Y$ means "X is followed by Y" (so a tweet by X is visible to Y). The team needs to (i) confirm reach, (ii) verify the visited-set is correctly used.



SUB	QUESTION	MARKS	CO	BTL
(a)	Run BFS from A bounded to depth 3 on the graph in the diagram. Show the BFS queue evolution as a table: (current node dequeued, depth, neighbours discovered, visited-set after this iteration). Process neighbours in alphabetical order on each level. Then state two things: (i) the final set of reached users (excluding A), (ii) for each user that has multiple in-edges from the diagram, name the predecessor chosen in your trace and explain why the visited-set prevents double-counting.	5	C03	BTL3
(b)	Complete the boilerplate below so that <code>bfs_bounded(adj, source, max_depth)</code> returns the set of all users reachable from source within <code>max_depth</code> hops, using a queue + visited set, in $O(V + E)$ for the visited subgraph. Note the return type and the pruning condition. <pre>import java.util.*; /** adj: {userId → list of users they're followed by}. * Returns set of user_ids reachable from `source` within `maxDepth` hops, * excluding `source` itself. */ static Set<String> bfsBounded(Map<String, List<String>> adj, String source, int maxDepth) { Set<String> visited = new HashSet<>(); visited.add(source); Deque<Object[]> queue = new ArrayDeque<>(); // each item = {user, depth} queue.offer(new Object[]{ source, 0 }); Set<String> reached = new HashSet<>(); while (!queue.isEmpty()) { Object[] cur = queue.poll(); String u = (String) cur[0]; int depth = (int) cur[1]; // TODO 1: if depth == maxDepth, do NOT expand neighbours // TODO 2: enumerate neighbours; add unseen ones to visited + queue with depth+1 // TODO 3: accumulate every newly-visited (non-source) user into `reached` } return reached; }</pre>	5	C03	BTL3
(c)	Capacity analysis under the 500 ms p99 budget. The author has 50 000 followers (depth-1); average follower has 500 followers (depth-2 candidates); etc. 1. Estimate the worst-case number of <i>distinct</i> users reached at depth ≤ 3 if the visited-set works correctly. (You may treat overlap as ~30 % of candidate edges going to already-visited users.) 2. Estimate the worst-case work without the visited-set (naive BFS that re-enqueues seen users). 3. State whether the worst-case visited-version is feasible at 500 ms p99. Use the assumption: 1 graph-neighbour visit $\approx 2 \mu\text{s}$ of work (memory random-access + visited-set check + enqueue).	5	C03	BTL5